

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [3] – [Konsep Dasar Pemrograman Berorientasi Objek]



NAMA: Lintang Anissa Hakim

NIM: 103132400040

KELAS: S1-SD-05

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1. Pemrograman Berorientasi Objek (OOP)

Pemrograman berorientasi objek merupakan metode pemrograman yang mengenkapsulasi data dan fungsi ke dalam kelompok yang dapat saling berinteraksi. Dalam OOP terdapat beberapa karakteristik utama, yaitu abstraksi, enkapsulasi, inheritance, polymorphism, dan komunikasi antar objek.

2. Enkapsulasi

Enkapsulasi merupakan proses pemaketan data objek bersama method-methodnya. Salah satu manfaatnya adalah menyembunyikan rincian implementasi dari objek lain yang tidak berhak mengaksesnya. Dalam penerapannya, data dapat dibuat `private` sehingga akses terhadap data dikendalikan melalui method yang tersedia.

3. Class dan Object

Class merupakan cetak biru atau rancangan dari sebuah objek. Sebuah class dapat memiliki anggota berupa atribut dan method. Object merupakan hasil instansiasi atau hasil ciptaan dari class yang telah dibuat sebelumnya. Dalam Java, object dapat dibuat menggunakan keyword `new`.

4. Field

Field merupakan atribut atau anggota dari suatu class yang digunakan untuk menyimpan data. Terdapat dua jenis field yang dijelaskan pada modul, yaitu **class field** dan **instance field**. Class field menggunakan modifier `static`, sedangkan instance field merupakan field yang tidak menggunakan modifier `static` dan setiap objek memiliki salinan field-nya sendiri.

5. Method

Method merupakan function atau procedure yang digunakan untuk memodularisasi program melalui pemisahan tugas dalam suatu class. Method dapat menerima parameter dan dapat mengembalikan nilai. Method yang mengembalikan nilai menggunakan tipe data `return`, sedangkan method yang tidak mengembalikan nilai menggunakan `void`.

6. Pass by Value dan Pass by Reference

Pada pass by value, nilai dari tipe data primitif disalin ketika diberikan kepada method sehingga perubahan pada parameter method tidak mengubah variabel asal. Sementara itu, array dan object berkaitan dengan referensi sehingga perubahan terhadap objek/array yang sama dapat terlihat pada variabel asal.

7. Keyword `this`

Keyword `this` merupakan referensi khusus yang menunjuk pada objek yang sedang digunakan. Keyword ini dapat digunakan untuk membedakan field dengan parameter method yang memiliki nama sama. Selain itu, `this` juga dapat digunakan untuk memanggil constructor lain dari dalam constructor.

B. Guided

1. [Abstraksi]

```
public class Abstraksi {  
    public static void main(String[] args) {  
        Kucing aren = new Kucing();  
  
        aren.nama = "Aren Gentong";  
        aren.ras = "Mujaer";  
        aren.bersuara();  
    }  
}
```

Output :

Aren GentongMujaermeow

2. [Kucing]

```
public class Kucing {  
  
    String nama;  
    String ras;  
  
    void bersuara() {  
        System.out.println(nama + ras + "meow");  
    }  
}
```

Output :

[masukkan screenshot output]

3. [Buku]

```
public class Buku {  
    private String pengarang;  
    private String judul;  
  
    private Buku() {  
        this("Rumah Kita", "GoodBles");  
        // this ini digunakan untuk memanggil konstruktor yang  
        // menerima dua parameter  
    }  
  
    private Buku(String judul, String pengarang) {  
        this.judul = judul;  
        this.pengarang = pengarang;  
    }  
}
```

```

        private void cetakKeLayar() {
            System.out.println("Judul : " + judul + " Pengarang : "
                               + pengarang);
        }

        public static void main(String[] args) {
            Buku a, b;
            a = new Buku("Jurassic Park", "Michael Chricton");
            b = new Buku();
            a.cetakKeLayar();
            b.cetakKeLayar();
        }
    }
}

```

Output :

```

Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles

```

4. [Circle]

```

public class Circle {

    public static final double PI = 3.14159;

    public static double radiansToDegrees(double rads) {
        return rads * 180 / PI;
    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }
}

```

Output :

[masukkan screenshot output]

5. [Demo Lagu]

```

class Lagu {

    private String pencipta;
    private String judul;
}

```

```

    public void IsiParam(String judul, String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }

    public void cetakKeLayar() {
        if (judul == null && pencipta == null) {
            return;
        }
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}

public class DemoLagu {

    public static void main(String[] args) {
        Lagu a = new Lagu();
        a.IsiParam("God Will Make A Way", "Don Moen ");
        a.cetakKeLayar();
    }
}

```

Output :

```

Judul : God Will Make A Way, pencipta : Don Moen

```

6. [Rekening]

```

public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah) {
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo() {
        System.out.println("Saldo Anda: " + saldo);
    }
}

```

Output :

7. [Test Pass]

```

public class TestPass {

    // 1. Atribut

```

```

int i, j;

// 2. Konstruktor
TestPass(int a, int b) {
    i = a;
    j = b;
}

// 3. Method Pass by Value (primitif)
void calculate(int m, int n) {
    m = m * 10;
    n = n / 2;
}

// 4. Method Pass by Reference (objek)
void calculate(TestPass e) {
    e.i = e.i * 10;
    e.j = e.j / 2;
}

// 5. Entry point untuk menjalankan program
public static void main(String[] args) {
    int x, y;
    TestPass z;
    z = new TestPass(50, 100);
    x = 10;
    y = 20;

    System.out.println("Nilai sebelum passed by value : ");
    System.out.println("x = " + x);
    System.out.println("y = " + y);

    // passed by value
    z.calculate(x, y);
    System.out.println("Nilai sesudah passed by value : ");
    System.out.println("x = " + x);
    System.out.println("y = " + y);

    System.out.println("Nilai sebelum passed by reference : ");
    System.out.println("z.i = " + z.i);
    System.out.println("z.j = " + z.j);

    // passed by reference
    z.calculate(z);
    System.out.println("Nilai sesudah passed by reference : ");
    System.out.println("z.i = " + z.i);
    System.out.println("z.j = " + z.j);
}
}

```

Output :

```
Nilai sebelum passed by value :  
x = 10  
y = 20  
Nilai sesudah passed by value :  
x = 10  
y = 20  
Nilai sebelum passed by reference :  
z.i = 50  
z.j = 100  
Nilai sesudah passed by reference :  
z.i = 500  
z.j = 50
```

8. [Encapsulation]

```
public class Encapsulation {  
  
    public static void main(String[] args) {  
        Rekening rek = new Rekening();  
  
        rek.tambahSaldo(20000000);  
        rek.tampilkanSaldo();  
    }  
}
```

Output :

```
Saldo berhasil ditambahkan  
Saldo Anda: 20000000
```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah. Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun – bukan sesuatu yang berbeda-beda tiap kali data dibuat – sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya. Buatlah satu class yang bertugas mengolah data suhu ini – memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Output yang diharapkan:

=== Data Suhu Awal ===

Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===

Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:

[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]

(ikut berubah: constructor menyimpan referensi array yang sama)

...

Jawaban :

```
public class Main {  
  
    public static void main(String[] args) {  
  
        // Data suhu mentah selama 7 hari  
        double[] suhuHarian = {  
            30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9  
        };  
  
        // Membuat object PengolahSuhu  
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);  
  
        // Menampilkan data awal  
        System.out.println("=== Data Suhu Awal ===");  
        pengolah.tampilkanData();  
  
        // Mencari index data yang kosong  
        int indexKosong = pengolah.cariIndexKosong();  
  
        System.out.println();  
        System.out.println(  
            "Index hari kosong (dimulai dari 0): " + indexKosong  
        );  
    }  
}
```

```

// Mengisi data yang kosong
pengolah.isiDataKosong();

// Menampilkan data setelah pengisian
System.out.println();
System.out.println("=== Data Suhu Setelah Pengisian ===");
pengolah.tampilkanData();

// Menghitung dan menampilkan rata-rata
double rataRata = pengolah.hitungRataRata();

System.out.printf(java.util.Locale.US, "%nRata-rata : %.2f\u00B0C%n",
rataRata);

// Menampilkan kembali array yang ada di main
System.out.println();
System.out.println(
    "Isi array suhuHarian di main setelah isiDataKosong() dijalankan:"
);
System.out.println(Arrays.toString(suhuHarian));
System.out.println("(ikut berubah: constructor menyimpan referensi array
yang sama)");
/*
 * Array suhuHarian di main ikut berubah karena array merupakan
 * tipe data reference.
 *
 * Saat object PengolahSuhu dibuat, constructor tidak membuat
 * array baru, tetapi menyimpan referensi ke array suhuHarian
 * yang sama di main melalui:
 *
 * this.suhuHarian = suhuHarian;
 *
 * Oleh karena itu, ketika isiDataKosong() mengubah isi array
 * melalui object PengolahSuhu, perubahan tersebut juga terlihat
 * pada variabel suhuHarian yang berada di main.
 */
}
}

```

Output :

```
=== Data Suhu Awal ===
```

```
Hari 1 : 30.4°C
```

```
Hari 2 : 24.3°C
```

```
Hari 3 : 26.8°C
```

```
Hari 4 : (kosong)
```

```
Hari 5 : 31.4°C
```

```
Hari 6 : 30.8°C
```

```
Hari 7 : 32.9°C
```

```
Index hari kosong (dimulai dari 0): 3
```

```
=== Data Suhu Setelah Pengisian ===
```

```
Hari 1 : 30.4°C
```

```
Hari 2 : 24.3°C
```

```
Hari 3 : 26.8°C
```

```
Hari 4 : 29.1°C
```

```
Hari 5 : 31.4°C
```

```
Hari 6 : 30.8°C
```

```
Hari 7 : 32.9°C
```

```
Rata-rata : 29.39°C
```

```
Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
```

```
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
```

```
(ikut berubah: constructor menyimpan referensi array yang sama)
```

Penjelasan :

Program ini digunakan untuk mengolah data suhu selama 7 hari menggunakan array suhuHarian. Nilai -1.0 menunjukkan bahwa data suhu pada hari ke-4 masih kosong. Setelah object PengolahSuhu dibuat, program menampilkan data awal dan mencari posisi data yang kosong. Hasilnya adalah index 3, yang berarti hari ke-4 karena index array dimulai dari 0.

Selanjutnya, data kosong diisi menggunakan rata-rata suhu hari sebelumnya dan sesudahnya, yaitu $(26.8 + 31.4) / 2 = 29.1^{\circ}\text{C}$. Setelah itu, program menghitung rata-rata seluruh suhu dan mendapatkan hasil **29.39°C**.

Array pada main juga ikut berubah karena object PengolahSuhu menggunakan **referensi array yang sama**. Jadi, perubahan data melalui object juga mengubah array asli. Program ini menerapkan konsep **class, object, constructor, method, encapsulation, this, dan reference pada array**.

.

2. Blueprint

Jawaban :

```
public class PengolahSuhu {
```

```
// Instance field
```

```

private double[] suhuHarian;

// Class field: nilainya tetap dan digunakan bersama semua object
public static final double NILAI_KOSONG = -1.0;

// Constructor
public PengolahSuhu(double[] suhuHarian) {
    this.suhuHarian = suhuHarian;
}

// Menampilkan data suhu
public void tampilkanData() {
    for (int i = 0; i < suhuHarian.length; i++) {

        if (suhuHarian[i] == NILAI_KOSONG) {
            System.out.println("Hari " + (i + 1) + " : (kosong)");
        } else {
            System.out.println(
                "Hari " + (i + 1) + " : " + suhuHarian[i] + "\u00B0C"
            );
        }
    }
}

// Mencari index data yang kosong
public int cariIndexKosong() {
    for (int i = 0; i < suhuHarian.length; i++) {

        if (suhuHarian[i] == NILAI_KOSONG) {
            return i;
        }
    }

    return -1;
}

// Mengisi data yang kosong menggunakan rata-rata
// suhu hari sebelum dan sesudahnya
public void isiDataKosong() {

    int indexKosong = cariIndexKosong();

    if (indexKosong != -1) {
        suhuHarian[indexKosong] =
            (suhuHarian[indexKosong - 1]
            + suhuHarian[indexKosong + 1]) / 2;
    }
}

// Menghitung rata-rata suhu
public double hitungRataRata() {

```

```
        double total = 0;

        for (double suhu : suhuHarian) {
            total += suhu;
        }

        return total / suhuHarian.length;
    }
}
```

Penjelasan :

Class PengolahSuhu digunakan untuk mengolah data suhu harian. suhuHarian merupakan **instance field** untuk menyimpan data suhu, sedangkan NILAI_KOSONG digunakan sebagai penanda data yang kosong dengan nilai -1.0.

Constructor digunakan untuk memasukkan data suhu ke dalam object. Method tampilkanData() menampilkan suhu setiap hari, cariIndexKosong() mencari posisi data yang kosong, dan isiDataKosong() mengisi data tersebut berdasarkan rata-rata suhu hari sebelumnya dan sesudahnya. Sementara itu, hitungRataRata() digunakan untuk menghitung rata-rata seluruh suhu.

Program ini menerapkan konsep **field, class field, constructor, method, private, static, dan this**.

D. Kesimpulan

- E. Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep dasar **Object-Oriented Programming (OOP)** pada Java dapat diterapkan melalui pembuatan class dan object. Pada praktikum ini, class PengolahSuhu digunakan untuk mengolah data suhu harian dengan menerapkan **field, constructor, method, encapsulation, static, dan this**. Program juga berhasil mencari data yang kosong, mengisinya berdasarkan rata-rata suhu di sekitarnya, serta menghitung rata-rata keseluruhan data. Praktikum ini membantu memahami bagaimana data dan fungsi dapat digabungkan dalam sebuah class sehingga program menjadi lebih terstruktur dan mudah dikelola.